4 Externalized configuration management

This chapter covers

- Configuring Spring with properties and profiles
- Applying external configuration with Spring Boot
- Implementing a configuration server with Spring Cloud Config Server
- Configuring applications with Spring Cloud Config Client

In the previous chapter, we built a RESTful application for managing a catalog of books. As part of the implementation, we defined some data to configure certain aspects of the application (in an application.yml file), such as the Tomcat thread pool and connection timeout. The next step might be to deploy the application to different environments: first in a test environment, then staging, and finally in production. What if you needed a different Tomcat configuration for each of these environments? How would you achieve that?

Traditional applications were usually packaged as a bundle, including the source code and a series of configuration files containing data for different environments, with the appropriate configuration being selected through a flag at runtime. The implication was that you had to make a new application build every time you needed to update the configuration data for a specific environment. A variant of this process was to create a different build for each environment, meaning that you had no guarantee whether what you ran in a staging environment would work the same way in production because they were different artifacts.